

Macro con Agentes Heterogeneos. Tarea 2

Fecha de entrega: 22 de Agosto de 2020

Esta tarea consiste en una serie de ejercicios para empezar a programar.

Ejercicio 1. Básicos Cree un programa que desarrolle los siguientes items

1. Cree un vector de dimensión $N \times 1$ con $N = 10$ y llámelo A, con valores igualmente distribuidos en el intervalo $[0.5, 10]$. (Tips: algunos lenguajes como Matlab tienen una función que lo hace. En otros lenguajes como fortran se lo puede hacer con un loop. En Matlab sería un loop "for", en fortran un loop "do"). Guarde ese vector como un archivo .txt con el nombre vector1.
2. Cree una matriz de dimensión $N \times N$ donde cada elemento sea igual a 1 y llámela B.
3. Genere una matriz C que sea la multiplicación del vector A elemento por elemento con la matriz B. De esta operación debería obtener una matriz $N \times N$ donde cada columna sea igual al vector creado en 1. Guarde esta matriz en un archivo que se llame matriz.txt. (Tip: los distintos lenguajes tienen distintos comandos para realizar estas operaciones, por ejemplo Matlab considera cada elemento como una matriz mientras que Fortran no lo hace).
4. Genere un vector D que sea la multiplicación del vector A traspuesto por la matriz B. En este caso debería obtener un vector de dimensión $(1 \times N)$ donde cada elemento sea la suma de los elementos de A.
5. Genere un vector de dimensión $N \times 1$ de números aleatorios, llámelo E (Tip: tenga en cuenta que los generadores de números aleatorios usan distintos algoritmos y muchas veces este algoritmo genera números distintos cada vez que se ejecuta el programa. En general si se quiere generar la misma secuencia de números hay que utilizar una semilla "seed").
6. Encuentre el valor máximo del vector E y la posición donde está ese valor.

Ejercicio 2. Tirada de un dado Este ejercicio pretende simular los resultados de tirar un dado.

1. Genere un vector de dimensión $N \times 1$ con $N = 6$ y llámelo "dado". Asigne el valor de 0 a todos los elementos.
2. Genere un loop, que vaya desde el valor 1 hasta $N_{tiradas}$. Vamos a simular distintos número de tiradas. Para el primer caso suponga que $N_{tiradas} = 100$.

- En cada vuelta del loop, genere un número aleatorio entre 0 y 1. Ahora usaremos un loop "if". Este tipo de loops son condicionales. Lo que haremos es asignar el valor dependiendo del valor del número aleatorio. En este caso tendremos varias condiciones, el programa debería tener una estructura como la siguiente:

```
if num aleatorio .leq. 1/6
    dado(1,1)=dado(1,1)+1
elseif num aleatorio .gt. 1/6 .and. numaleatorio .leq. 2/6
    dado(2,1)=dado(2,1)+1
...
elseif num aleatorio .gt. 5/6
    dado(6,1)=dado(6,1)+1
end
```

Note que como dado tiene valores 0 al inicio lo que estamos haciendo es ir contando las veces que el número aleatorio se encuentra en cada intervalo. Si el dado es justo debería tener 1/6 de probabilidad de sacar cada número y lo que estamos haciendo es asignar cada intervalo de probabilidad a un número.

- Guarde el vector dado como .txt. Cree un do-file en stata que lea este archivo y genere un histograma con los resultados
3. Repita el proceso para $N_{tiradas} = 1000$ y para $N_{tiradas} = 10000$. Describa que observa en cada caso.

Ejercicio 3. Discretización de un proceso de Markov.

(Descripción del libro de Heer y Maussner: Dynamic General Equilibrium Modelling pag. 657 y 658)

En gran parte de los ejercicios que vamos a resolver supondremos que existe un proceso de Markov para los shocks que reciben los agentes. En este ejercicio veremos como discretizar

un proceso estadístico. Suponga que queremos discretizar el siguiente proceso estadístico $z_t = \rho z_{t-1} + \epsilon_t$ donde $\epsilon_t \sim N(0, \sigma_\epsilon^2)$. Tauchen propone construir grids equidistantes para z_t y asignar probabilidades a cada intervalo dado que sabemos que cada realización z_i de la variable z_t se distribuye Normal con media ρz_i y varianza σ_ϵ^2 . Para definir z_1 y z_m vamos a usar una variable λ que nos dice a cuántas desviaciones estándar vamos a truncar la distribución. Como el vector tiene puntos equidistantes, podemos definir $dz = \frac{z_{i+1} - z_i}{2}$.

En este caso podemos calcular la probabilidad de que z esté en el intervalo $z_i - dz$ y $z_i + dz$ como $prob(z_i - dz \leq z \leq z_i + dz) = \pi(z_i + dz) - \pi(z_i - dz)$, donde $\pi(\cdot)$ es la función de distribución acumulada de la distribución normal. La probabilidad de pasar de un valor z_i a otro z_j es el área bajo la curva de densidad en el intervalo $\left[\frac{z_j - \rho z_i - dz}{\sigma_\epsilon}, \frac{z_j - \rho z_i + dz}{\sigma_\epsilon}\right]$. La probabilidad de estar en el estado z_1 es el área bajo la curva de densidad en el intervalo $[-\infty, z_1 + dz]$ y la del último punto es $p_{im} = 1 - \sum_{j=1}^{m-1} p_{ij}$.

1. Discreticemos los valores de z . Definimamos a $z_1 = -\lambda\sigma_\epsilon/\sqrt{1-\rho^2}$, $z_m = -z_1$. Elija el salto entre cada punto como $salto = -2z_1/(m-1)$, y para cada $m = 2, \dots, m-1$ compute $z_i = z_1 + (i-1) * salto$
2. Calculamos la función de transición $P = (p_{ij})$, sea $\pi(\cdot)$ sea la distribución acumulada de una Normal estándar. Para $i = 1, 2, \dots, m$ asigne:

$$\begin{aligned}
 p_{i1} &= \pi\left(\frac{z_1 - \rho z_i}{\sigma_\epsilon} + \frac{salto}{2\sigma_\epsilon}\right) \\
 p_{ij} &= \pi\left(\frac{z_j - \rho z_i}{\sigma_\epsilon} + \frac{salto}{2\sigma_\epsilon}\right) - \pi\left(\frac{z_j - \rho z_i}{\sigma_\epsilon} - \frac{salto}{2\sigma_\epsilon}\right) \\
 j &= 2, 3, \dots, m-1 \\
 p_{im} &= 1 - \sum_{j=1}^{m-1} p_{ij}
 \end{aligned}$$

Siguiendo estos pasos escriba un programa que:

1. Calcule el vector de shocks z y la matriz de transición P para el proceso autoregresivo con parámetros:
 $\lambda = 3$, $\sigma_\epsilon^2 = 0.007$, $n_z = 5$ y $\rho = 0.9$.
(Nota: algunos lenguajes tienen como función incorporada la distribución normal, mientras que en otros hay que descargarse alguna rutina que lo discretice)

2. Calcule el vector de la distribución invariante. Para esto realice los siguientes pasos:

- Genere un vector de dimensión $1 \times M$ y llámelo $guess_{inv}$, donde cada elemento contenga el valor $1/m$.
- Realice un loop while que tenga como condición $error > tol$ y que $iter < itermax$. Este tipo de loops continúan el proceso hasta que se satisface alguna condición. En este caso la condición que imponemos es que el error entre inv y $guess_{inv}$ sea menor a una tolerancia. $|\max(inv - guess_{inv})|$, esto es el valor máximo de la diferencia entre estos 2 vectores. El vector inv se calcula de la siguiente manera:
$$inv = guess_{inv} * P$$

si la diferencia es mayor actualice a $guess_{inv} = inv$ y vuelva al inicio. En realidad el loop while volverá al inicio porque no se cumplió la condición.

Use valores para $tol = 0.00001$ y para $itermax = 500$